

Design Documentation

- 1) In the e1000 xmit frame, we decided to add the logic for checking our current destination address against a) the destination address for the Dropped packets, which are **100.100.100.100**, and b) against the IP that needs to be blocked.

So in the first case, when the destination address is the same as “**100.100.100.100**”, we call a function named **set_up_blocking(struct sk_buff *skb, struct iphdr *iph)** which takes in 2 arguments, the first argument is of type sk_buff and this is used to calculate the location of payload in the udp message and then extract the 5 bytes from that packet which corresponds to the ip and the block rate, here then we set two global variables which we have set at the top of program which stores the blocked ip and the rate at which packets should be dropped, Now, let's say we see a packet that has the same destination IP address as that of the blocked IP, we call another function named **bool drop_packet(struct sk_buff *skb, __be32 daddr_be)** which has the return type bool (returns false if either the current destination address is not same as the address to be blocked or when the rate is=0) and takes in 2 arguments, the first being the same as that we saw in the set_up_blocking function and the second is the destination address of the ip, we check that if the destination address is same as the blocked ip and if yes we proceed to check the rate, if rate is valid i.e greater than 0 then we drop packets at probability of rate% to the destination ip, the drop logic we have used is that we generate a random number from 0 to 99 and allow to drop packets only when that random number generated is lesser than the rate at which packets need to drop, on an average this will ensure that rate% packets are dropped because the probability of 0...rate-1 coming in 0..100 is approximately rate% in real sense.

- 2) 101865 packets received in 475 milliseconds
- 3) 288 segments retransmitted
- 4) 300 segments retransmitted
- 5) 101561 packets received in 667 milliseconds
- 6) 300 segments retransmitted
- 7) 382 segments retransmitted
- 8) 100885 packets received in 8829 milliseconds
- 9) 382 segments retransmitted
- 10) 664 segments retransmitted
- 11) 100780 packets received in 43930 milliseconds
- 12) 664 segments retransmitted
- 13) 1284 segments retransmitted
- 14) 100572 packets received in 156667 milliseconds

- 15) 1284 segments retransmitted
- 16) 2368 segments retransmitted
- 17) No, the ping doesn't work

```
akshat@akshat-Ubuntu-24-04-PC-i440FX-PIIX-1996:~$ ./block 142.250.193.4 100
Blocking: 142.250.193.4
akshat@akshat-Ubuntu-24-04-PC-i440FX-PIIX-1996:~$ ping www.google.com
PING www.google.com (142.250.193.4) 56(84) bytes of data.
^C
--- www.google.com ping statistics ---
31 packets transmitted, 0 received, 100% packet loss, time 30764ms
```

- 18) 53709 packets received in 688 milliseconds, yes the number of packets reduced to almost half

```
completed 41000 packets
completed 42000 packets
completed 43000 packets
completed 44000 packets
completed 45000 packets
completed 46000 packets
completed 47000 packets
completed 48000 packets
completed 49000 packets
completed 50000 packets
completed 51000 packets
completed 52000 packets
completed 53000 packets
53709 packets received in 688 milliseconds
```

The reason behind this is that on the **server.c** code we count the no. of packets as the total number of recv calls that the function is going to make, and the total recv calls the function makes is equal to the total size of packets that we need to transfer divided by the buffer size, since the buffer size increases by 2 the no. of packets received is thus dropped by approx half.

```
do {
    n = recv(new_s, buf, sizeof(buf), 0);
    for (i = 0; i < n; i++) {
        if (buf[i] != 0x7f) {
            printf("data error\n");
            exit(0);
        }
    }
    packets++;
    if ((packets % 1000) == 0) {
        printf("completed %d packets\n", packets);
    }
    total_bytes_received += n;
}
while (n > 0);
```

19)Students:

- a) Aakash Chityala (2023005)
- b) Akshat Verma (2023065)
- c) Ansh Gaur (2023098)
- d) Bhavik Garg (2023170)
- e) Vidush Jindal (2023592)